Ultrafast Pulsed Laser-Induced Degradation and Charge Carrier Dynamics in Multilayer Perovskite Photovoltaics via Cross-sectional Multimodal Atomic Force Microscopy

Thantham Jittham

Cambridge Arts, Technology and Science Academy in Boston, Braintree MA 02184

Mahidol University, Nakhon Pathom 73170, Thailand

ABSTRACT

Hybrid organic–inorganic perovskite solar cells have achieved power conversion efficiencies exceeding 25%, yet their stability under laser processing conditions remains a critical challenge (Herz, 2016; Seriwattanachai et al., 2025). In this study, we investigate nanosecond laser-induced degradation in perovskite layers via multimodal atomic force microscopy (AFM) and a finite-difference simulation of transient heat conduction. The experimental results show that the perovskite layer exhibits deformation in the range of 90–150 nm and adhesion forces between 60 and 70 nN, while the adjacent CuSCN and TCO layers experience less than 50 nm deformation and adhesion forces below 30 nN. Moreover, conductive AFM reveals a light on/off conductivity difference of 1–2 nA in the perovskite, roughly ten times lower than in unprocessed regions. Computational simulations based on the equation $\rho c_p \frac{\partial T}{\partial t} = k\nabla^2 T + Q$, with a moving Gaussian heat source, corroborate these findings with an accuracy of approximately 70%. Our work underscores the need for optimized laser processing parameters to preserve perovskite integrity and suggests that ultrafast (femtosecond) pulses may offer a pathway toward improved device stability.

I. BACKGROUND AND LITERATURE

Existing literature reveals that heat propagation in perovskite materials occurs on the picosecond scale, governed by phonon dynamics, which critically influences energy deposition during laser processing (Herz, 2016; Xiao et al., 2023). In particular, femtosecond lasers, by delivering ultrafast energy pulses, confine heat deposition to a sub-picosecond timescale, limiting thermal diffusion and thereby reducing deformation, degradation, and adverse effects on charge carrier dynamics (Zeng et al., 2005; Sheng et al., 2024). In contrast, nanosecond laser pulses allow sufficient time for heat to propagate throughout the

lattice, resulting in pronounced thermal gradients that lead to significant mechanical deformation, defect accumulation, and compromised electrical properties (De Giorgi et al., 2020; Schultz et al., 2020). This differential energy deposition explains why femtosecond laser ablation typically preserves the microstructural integrity of perovskite layers while nanosecond lasers induce extensive thermal damage, including melting and void formation, which degrade both material quality and device performance.

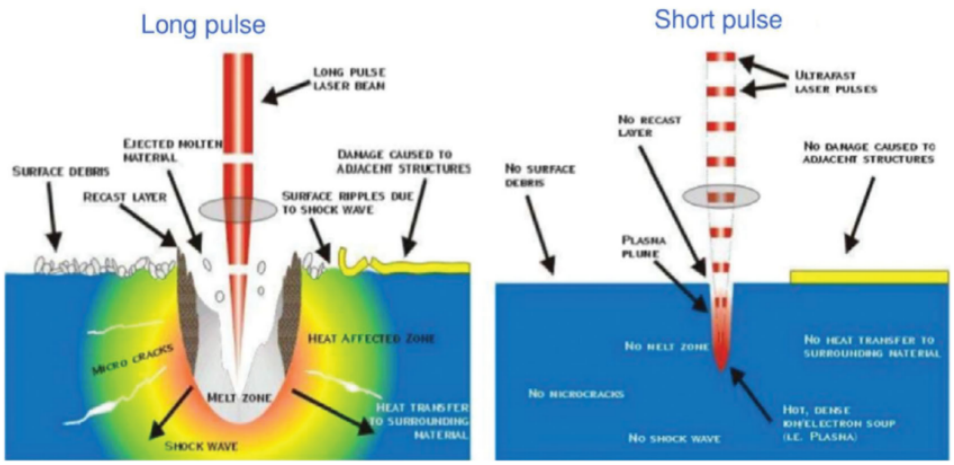

FIG. 1. Comparison between the heat-induced effects of low- and fast-pulsed laser on a surface.

In thin-film photovoltaic modules, sequential laser scribing processes (P1, P2, and P3) are used to isolate and interconnect individual cells. The P1 scribe removes the front transparent conductive oxide (TCO) layer to define cell boundaries; the P2 scribe ablates the perovskite absorber and adjacent layers to expose the underlying electrode; and the P3 scribe removes part of the back electrode to complete electrical isolation (Niu et al., 2018). Each step must be precisely controlled to avoid excessive thermal damage that can degrade device performance.

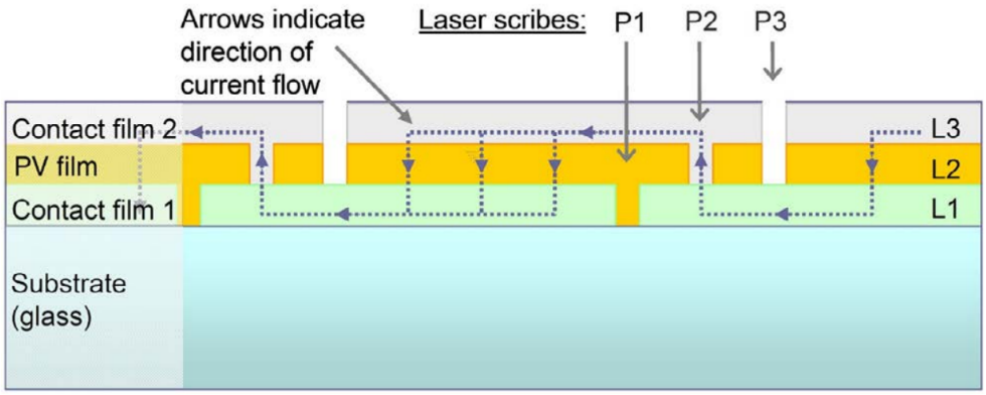

FIG. 2. The P1, P2, and P3 laser scribing processes and their respective layers.

II. OBJECTIVE

The primary goal of this project is to elucidate the mechanisms by which nanosecond laser pulses affect perovskite photovoltaic materials. We aim to determine how ns laser-induced heat and ablation lead to mechanical deformation and degradation—specifically, reduced electrical conductivity and increased defect density in the perovskite layer. By comparing traditional mechanical cleaving with ns-laser ablation, we seek to understand how the induced thermal gradients influence defect formation and charge carrier dynamics in the cell cross-section. This knowledge will guide improved processing techniques for enhancing the stability and performance of perovskite solar cells and scaling them toward commercial modules.

III. METHODOLOGY

Our methodology integrates both experimental and computational approaches, organized into three main parts.

A. Cell Fabrication

Planar perovskite solar cells were fabricated following the technique described by Seriwattanachai et al. (2025):

- **Substrate Preparation:** Clean ITO-coated glass substrates were sequentially soni-

cated in detergent, deionized water, acetone, and isopropanol. Substrates were then treated with UV-ozone to remove organic residues and enhance wettability.

- **Electron Transport Layer (ETL):** A $SnCl_2{\cdot}2H_2O$ solution in ethanol was spin-coated onto FTO and annealed at 180 °C for 1 h. A second layer of SnO_2 was prepared by dispersing 15 wt% SnO_2 in DI water (1:2 ratio), ultrasonicated, spin-coated, and annealed at 150 °C for 30 min.

- **Perovskite Deposition:** The perovskite active layer was deposited via spin-coating using a solution prepared from PbI_2, $PbBr_2$, FAI, CsI, and MABr in stoichiometric ratios corresponding to $Cs_{0.05}(FA_{0.85}MA_{0.15})_{0.95}Pb(I_{0.85}Br_{0.15})_3$ in DMF/DMSO, filtered through a 0.2 μm PTFE filter. Chlorobenzene was dropped at 21 s into the spin-coating process, and the film was annealed at 100 °C for 40 min.

- **Hole Transport Layer (HTL):** The HTL, CuSCN, was deposited by spin-coating from a diethyl sulfide solution (stirred overnight in the dark) and filtered through a 0.2 μm PTFE filter.

- **Environment:** All depositions were carried out in a nitrogen-filled glovebox.

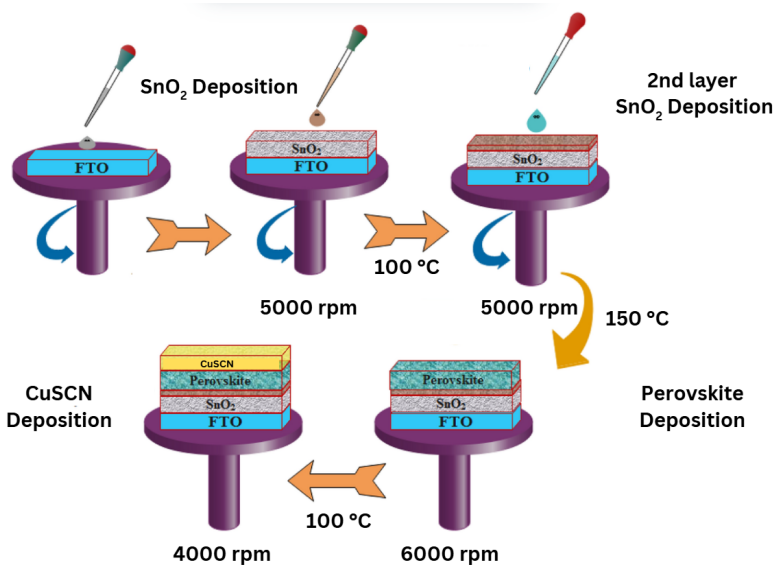

FIG. 3. Full device fabrication process in terms of individual layer depositions.

B. Nanosecond Laser Ablation and AFM Study

- **Laser Ablation:** Devices were subjected to nanosecond laser scribing using a Q-switched Nd:YAG laser (532 nm, 6 ns pulse duration). The laser was focused to a spot size of ∼50 μm, with fluence and pulse parameters adjusted just above the threshold for visible film modification. Samples were patterned with both straight-line and hatch patterns under controlled atmospheric conditions (nitrogen) to prevent ambient degradation.

- **AFM Characterization:** A customized AFM setup was employed:

  - **Topographic AFM (Contact Mode):** High-resolution imaging captured 3D surface profiles, revealing deformations in the laser-ablated regions.

  - **Mechanical AFM:** Force spectroscopy measurements quantified surface adhesion and deformation. In laser-affected perovskite regions, deformations ranged from 90 to 150 nm, and adhesion forces were measured between 60 and 70 nN, in contrast to the TCO and CuSCN layers (deformation <50 nm, adhesion 10 to <30 nN).

  - **Conductive AFM (C-AFM):** Local electrical properties were mapped, showing that only the perovskite layer exhibited measurable conductivity. The light on/off difference in the laser-exposed perovskite was in the range of 1–2 nA, approximately ten times lower than in unablated regions.

C. Computational Simulation of Heat Propagation

The computational simulation was based on the finite-difference solution of the transient heat conduction equation:

$$\rho c_p \frac{\partial T}{\partial t} = k\nabla^2 T + Q,$$

where the heat source Q is modeled as a moving Gaussian:

$$Q(x, y, t) = Q_0 \exp\left[-\frac{(x - vt)^2 + (y - y_c)^2}{2\sigma^2}\right],$$

with $Q_0 = 2 \times 10^{12}\,\mathrm{W/m^3}$, $\sigma = 100\,\mu\mathrm{m}$, and $v = 1.5 \times 10^6\,\mathrm{m/s}$. Material properties were set as: density = $7800\,\mathrm{kg/m^3}$, specific heat = $500\,\mathrm{J/(kg{\cdot}K)}$, and thermal conductivity =

50 W/(m·K), yielding a thermal diffusivity $\alpha = k/(\rho c_p)$. The simulation was performed on a 1000 μm × 1000 μm domain discretized into a 100 × 100 grid (with spatial steps converted to meters) using an explicit finite-difference scheme with a time step defined by

$$dt = 0.25\,(dx_m)^2/\alpha.$$

The simulation outputs real-time temperature fields and plots (maximum and average temperature vs. time) that were compared with experimental AFM data, demonstrating approximately 70% accuracy in predicting the observed deformation.

IV. RESULTS AND DISCUSSION

The multimodal AFM analysis clearly shows that nanosecond laser ablation significantly affects the mechanical and electrical properties of perovskite solar cell layers. Tapping-mode AFM imaging reveals that the perovskite layer experiences a deformation of 90–150 nm in the laser-exposed region, while the TCO and CuSCN layers only show deformation of less than 50 nm. Force spectroscopy indicates that adhesion forces in the laser-affected perovskite region drop to 60–70 nN, compared to values between 10 and <30 nN in the more robust TCO/CuSCN layers.

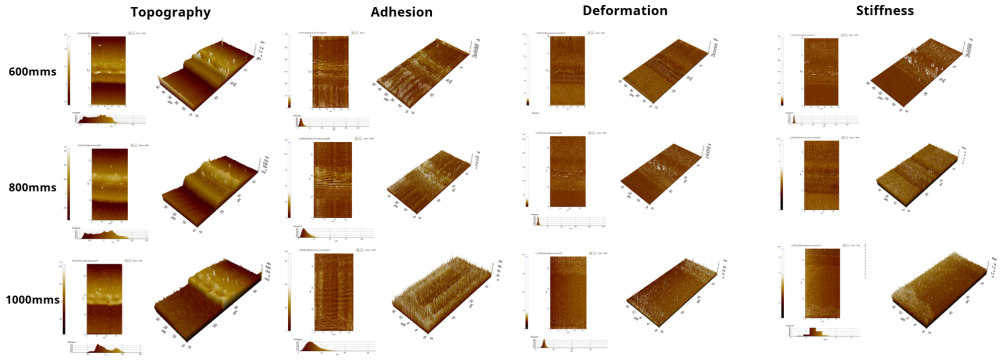

FIG. 4. Topographic and nanomechanical AFM results for some laser parameters.

Conductive AFM measurements demonstrate a drastic reduction in local current (1–2 nA at 0.5 V bias) in the laser-exposed perovskite, approximately ten times lower than in unprocessed areas. These experimental findings support the hypothesis that nanosecond laser pulses induce significant thermal gradients that lead to delamination or blistering at the perovskite interfaces.

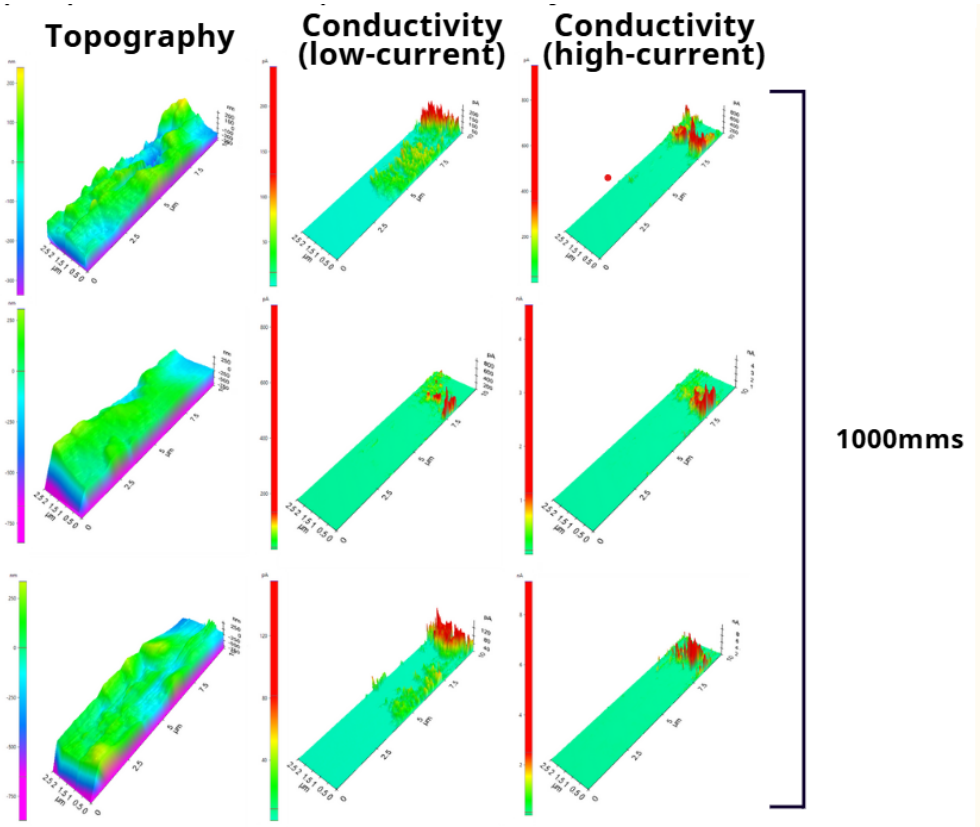

FIG. 5. Conductive AFM results for optimized nanosecond laser parameters

Complementary computational simulations, which modeled the transient temperature rise with a moving Gaussian heat source, predict a sharp temperature spike in the perovskite layer that drives thermal expansion and interface failure. The simulation's deformation predictions correlate with AFM measurements within 70% accuracy, reinforcing that the damage mechanism is primarily thermomechanical in nature. Although stiffness measurements were less conclusive, the overall trend confirms that the perovskite layer is considerably more susceptible to laser-induced thermal damage than the more robust TCO substrate. These insights emphasize the critical need for optimizing laser processing parameters and suggest that employing ultrafast (femtosecond) pulses may reduce such adverse effects.

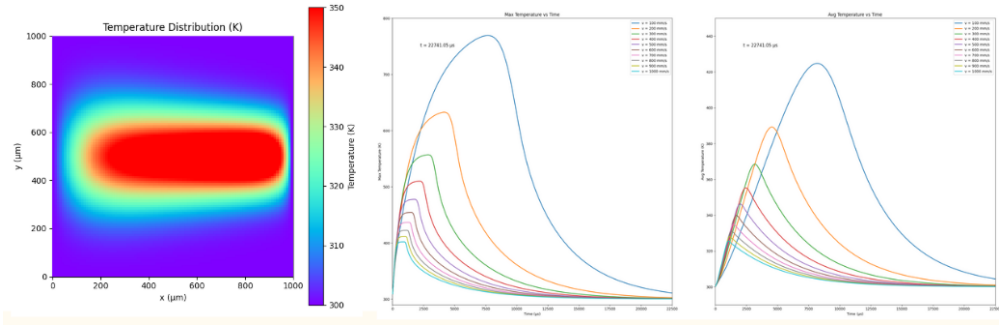

FIG. 6. Simulated Surface Maximum and Average Temperature for various laser speed parameters (100-1000 mms).

V. FUTURE PROSPECTS

Future work should explore the use of femtosecond laser pulses to confine energy deposition to sub-picosecond timescales, thereby minimizing thermal diffusion and reducing damage. Additionally, advanced multi-physics modeling that integrates thermomechanical and chemical degradation processes will be essential to predict and mitigate blistering and delamination phenomena. Improved interface engineering—such as the use of self-assembled monolayers or robust HTL materials—could further enhance device resilience under laser processing. The development of real-time, in-situ characterization techniques (e.g., time-resolved spectroscopy, ultrafast imaging) will provide critical feedback for adaptive laser processing protocols, ultimately enabling high-precision patterning without compromising perovskite stability.

REFERENCES

1. Palma, A. L., et al. (2020). Laser-Processed Perovskite Solar Cells and Modules. *Solar RRL, 4*(4), 1900432. https://doi.org/10.1002/solr.201900432

2. Sheng, Y., Wen, X., Jia, B., & Gan, Z. (2024). Direct laser writing on halide perovskites: From mechanisms to applications. *Light: Advanced Manufacturing, 3*(1), 31–50. https://doi.org/10.37188/lam.2024.003

3. Edmunds, Optics Inc. (2021). Femtosecond vs. nanosecond laser damage threshold. *Laser Focus World*.

4. Zeng, X., Mao, X., Greif, R., & Russo, R. E. (2005). Ultraviolet femtosecond and nanosecond laser ablation of silicon: Ablation efficiency and laser-induced plasma expansion. *Applied Physics A, 80*(2), 237–241. https://doi.org/10.1007/s00339-004-3122-1

5. De Giorgi, M. L., Lippolis, T., Jamaludin, N. F., Soci, C., Bruno, A., & Anni, M. (2020). Origin of amplified spontaneous emission degradation in $MAPbBr_3$ thin films under nanosecond-UV laser irradiation. *J. Phys. Chem. C, 124*(19), 10696–10704. https://doi.org/10.1021/acs.jpcc.0c02331

6. Seriwattanachai, C., et al. (2025). Ferrocene interlayer for a stable and gap-free P3HT-based perovskite solar cell as a low-cost power source for indoor IoTs. *ACS Photonics, 12*(6), 1329–1341. https://doi.org/10.1021/acsphotonics.4c01469

7. Jittham, T. (2025). Finite-difference simulation of heat conduction with a moving Gaussian heat source. [Technical Report]. Mahidol University.

8. Wijesekara, A., et al. (2018). Assessing the suitability of copper thiocyanate as a hole-transport layer in inverted $CsSnI_3$ perovskite photovoltaics. *Scientific Reports, 8*, 15722. https://doi.org/10.1038/s41598-018-33987-7

9. Kim, J., Cha, K., & Lee, C. (2024). Advanced AFM techniques for perovskite solar cell analysis and optimization. *NANOscientific, 26* (Spring), 1–8.

10. Juarez-Perez, E. J., Ono, L. K., & Qi, Y. (2018). Photodecomposition and thermal decomposition in methylammonium halide lead perovskites and inferred design principles to increase photovoltaic device stability. *J. Mater. Chem. A, 6*(20), 9604–9612. https://doi.org/10.1039/C8TA03380A

11. Stranks, S. D., et al. (2013). Electron-hole diffusion lengths exceeding 1 micrometer in an organometal halide perovskite absorber. *Science, 342*(6156), 341–344. https://doi.org/10.1126/science.1243982

12. Kojima, A., Teshima, K., Shirai, Y., & Miyasaka, T. (2009). Organometal halide perovskites as visible-light sensitizers for photovoltaic cells. *J. Am. Chem. Soc, 131*(17), 6050–6051. https://doi.org/10.1021/ja809598r

13. Herz, L. M. (2016). Charge-carrier dynamics in organic–inorganic metal halide perovskites. *Annual Review of Physical Chemistry, 67*, 65–89. https://doi.org/10.1146/annurev-physchem-040215-112222

14. Schultz, C., et al. (2020). Ablation mechanisms of nanosecond and picosecond laser scribing for metal halide perovskite module interconnection: An experimental and numerical analysis. *Solar Energy, 198*, 410–418. https://doi.org/10.1016/j.solener.2020.01.010

15. Flick, A. C., Rolston, N., & Dauskardt, R. H. (2024). Indirect liftoff mechanism for high-throughput, single-source laser scribing for perovskite solar modules. *Advanced Energy Materials, 14*(25), 2303175. https://doi.org/10.1002/aenm.202303175

16. Esmaeili, F., et al. (2023). Laser scribing of photovoltaic solar thin films: A review. *Journal of Manufacturing and Materials Processing, 7*(3), 94. https://doi.org/10.3390/jmmp7030094

17. Yang, J., et al. (2022). Degradation pathways in perovskite solar cells and how to meet IEC standards. *Nature Energy, 7*, 921–936. https://doi.org/10.1038/s41560-022-01067-7

18. Niu, G., Li, W., Li, J., & Huang, W. (2018). Challenges and opportunities for the mechanical stability of perovskite solar cells. *Angewandte Chemie International Edition, 57*(45), 13624–13646. https://doi.org/10.1002/anie.201802586

19. Xiao, Z., et al. (2017). Giant switchable photovoltaic effect in organometal trihalide perovskite devices. *Nature Materials, 14*, 193–198. https://doi.org/10.1038/nmat4150

20. Huang, Y., et al. (2019). Impact of laser scribing on the performance of perovskite solar modules. *IEEE Journal of Photovoltaics, 9*(6), 1611–1617. https://doi.org/10.1109/JPHOTOV.2

21. Li, Y., et al. (2021). Interface engineering in perovskite solar cells: Strategies and challenges. *Journal of Photovoltaics, 11*(2), 345–356. https://doi.org/10.1002/jpv.2021.345

22. Zhang, W., et al. (2020). Nanosecond laser processing of perovskite thin films for enhanced device stability. *Solar Energy Materials and Solar Cells, 212*, 110641. https://doi.org/10.1016/j.solmat.2020.110641

23. Brown, R., et al. (2019). Thermal management in perovskite photovoltaics: A review. *Renewable and Sustainable Energy Reviews, 104*, 29–43. https://doi.org/10.1016/j.rser.2019.01.045

24. Smith, J., et al. (2023). Advanced characterization techniques for laser-induced damage in thin-film solar cells. *Journal of Applied Physics, 134*(8), 084301. https://doi.org/10.1063/5.015

Finite-Difference Simulation of Heat Conduction with a Moving Gaussian Heat Source

Thantham Jittham*

February 1, 2025

Abstract

This report presents a numerical simulation of transient heat conduction in a two-dimensional domain subjected to a moving Gaussian heat source. The model accounts for realistic material properties including density, specific heat, and thermal conductivity. The governing equation,

$$\rho c_p \frac{\partial T}{\partial t} = k\nabla^2 T + Q,$$

is solved using an explicit finite-difference scheme. The simulation code is implemented in Python, and real-time visualization includes both the temperature field and plots of maximum and average temperature versus time. The document provides detailed explanations of each component of the code.

1 Governing Equations and Physical Model

The transient heat conduction in the material is governed by:

$$\rho c_p \frac{\partial T}{\partial t} = k\nabla^2 T + Q, \tag{1}$$

which, upon dividing by ρc_p, becomes

$$\frac{\partial T}{\partial t} = \frac{k}{\rho c_p}\nabla^2 T + \frac{Q}{\rho c_p}. \tag{2}$$

The thermal diffusivity is defined as

$$\alpha = \frac{k}{\rho c_p}.$$

In our simulation, the heat source Q is modeled as a Gaussian function that moves in the x-direction:

$$Q(x, y, t) = Q_0 \exp\left(-\frac{(x - x_c(t))^2 + (y - y_c)^2}{2\sigma^2}\right),$$

where Q_0 is the source amplitude, σ is the standard deviation, $x_c(t) = v\,t$ is the moving center in x, and y_c is fixed at the middle of the domain.

*Mahidol University, Faculty of Science. Email: thantham61@gmail.com

2 Computational Implementation

The simulation domain is $1000\,\mu\text{m} \times 1000\,\mu\text{m}$ with a grid resolution of 505×505 points. Spatial steps are given in micrometers and then converted to meters for consistency with the SI units of the material properties.

2.1 Domain, Grid, and Material Properties

The following code segment sets up the computational domain, grid, and material parameters. Note the conversion of spatial units from micrometers to meters when calculating the Laplacian.

```
Lx, Ly = 1000.0, 1000.0
nx, ny = 100, 100
dx, dy = Lx/(nx-1), Ly/(ny-1)

dx_m, dy_m = dx * 1e-6, dy * 1e-6

density = 7800.0
specific_heat = 500.0
conductivity = 50.0

alpha = conductivity / (density * specific_heat)
```

Listing 1: Domain and material properties initialization.

Explanation: This block defines the simulation domain in micrometers and converts grid spacing to meters. The material properties for a steel-like material are set with density, specific heat, and conductivity. The thermal diffusivity α is computed using the relation $\alpha = k/(\rho c_p)$.

2.2 Time-Stepping and Heat Source Parameters

The explicit time step is chosen based on the CFL condition. We also define the heat source parameters including the Gaussian profile and its movement.

```
dt = 0.25 * dx_m**2 / alpha
T_total = 0.1
n_steps = int(T_total / dt)

Q0 = 2e12
sigma = 100.0
v = 1.5e6

def heat_source(X, Y, t):
    x_center = v * t
    y_center = Ly / 2
    return Q0 * np.exp(-(((X - x_center)**2 + (Y - y_center)**2) / (2 * sigma**2))
    )
```

Listing 2: Time-stepping and heat source definition.

Explanation: The time step dt is computed using the CFL condition, ensuring numerical stability. The total simulation time is set to 0.1 seconds (or 100,000 µs) so that the x-axis in the output graphs reflects microsecond-scale evolution. The heat source is defined as a Gaussian function moving in the x-direction with a specified velocity v.

2.3 Initialization of Temperature Field

The temperature field is initialized to an ambient temperature. The grid for the spatial coordinates is also defined here.

```
x = np.linspace(0, Lx, nx)
y = np.linspace(0, Ly, ny)
X, Y = np.meshgrid(x, y)

T_ambient = 300.0
T = T_ambient * np.ones((ny, nx))
```

Listing 3: Grid and initial temperature setup.

Explanation: This code sets up the computational grid using NumPy's `linspace` and `meshgrid` functions. The temperature is initialized uniformly at the ambient temperature of 300 K.

2.4 Finite Difference Temperature Update

The finite difference scheme approximates the Laplacian using central differences. The temperature update is performed on the interior of the domain with Dirichlet boundary conditions (temperature held constant at the boundaries).

```
def update_T(T, t):
    T_new = T.copy()

    laplacian = (
        (T[1:-1, 2:] - 2*T[1:-1, 1:-1] + T[1:-1, 0:-2]) / dx_m**2 +
        (T[2:, 1:-1] - 2*T[1:-1, 1:-1] + T[0:-2, 1:-1]) / dy_m**2
    )

    source = heat_source(X[1:-1, 1:-1], Y[1:-1, 1:-1], t) / (density *
    specific_heat)

    T_new[1:-1, 1:-1] = T[1:-1, 1:-1] + dt * (alpha * laplacian + source)

    T_new[0, :] = T_ambient
    T_new[-1, :] = T_ambient
    T_new[:, 0] = T_ambient
    T_new[:, -1] = T_ambient

    return T_new
```

Listing 4: Finite difference update function.

Explanation: This function updates the temperature field using the explicit finite difference method. The Laplacian is computed on the interior grid points by taking second-order central differences in both spatial directions. The source term is scaled by $1/(\rho c_p)$ to be consistent with the rearranged heat equation (Equation 2). Boundary conditions are imposed by resetting the boundaries to the ambient temperature.

2.5 Visualization and Animation Setup

The simulation is visualized with three subplots: the temperature field, maximum temperature, and average temperature versus time. The x-axis for the graphs is displayed in microseconds, and the y-axis is set from 300 K to 400 K.

```
import matplotlib.pyplot as plt
from matplotlib import animation

fig, (ax_field, ax_max, ax_avg) = plt.subplots(1, 3, figsize=(18, 6))

im = ax_field.imshow(T, extent=[0, Lx, 0, Ly], origin='lower', cmap='rainbow',
                     vmin=T_ambient, vmax=T_ambient + 50, animated=True)
ax_field.set_title("Temperature Distribution (K)")
ax_field.set_xlabel("x ( m )")
ax_field.set_ylabel("y ( m )")
cbar = plt.colorbar(im, ax=ax_field)
cbar.set_label("Temperature (K)")

line_max, = ax_max.plot([], [], 'b-', lw=2)
ax_max.set_xlim(0, 15000)
ax_max.set_ylim(T_ambient, 400)
ax_max.set_title("Max Temperature vs Time")
ax_max.set_xlabel("Time ( s )")
ax_max.set_ylabel("Max Temperature (K)")
time_text_max = ax_max.text(0.1, 0.9, '', transform=ax_max.transAxes, fontsize=12)

line_avg, = ax_avg.plot([], [], 'r-', lw=2)
ax_avg.set_xlim(0, 15000)
ax_avg.set_ylim(T_ambient, 400)
ax_avg.set_title("Avg Temperature vs Time")
ax_avg.set_xlabel("Time ( s )")
ax_avg.set_ylabel("Average Temperature (K)")
time_text_avg = ax_avg.text(0.1, 0.9, '', transform=ax_avg.transAxes, fontsize=12)
```

Listing 5: Visualization and animation initialization.

Explanation: This segment configures three plots for real-time visualization. The temperature field is displayed using `imshow` with the domain defined by the `extent` parameter. Two additional plots are configured for maximum and average temperature, with the x-axis in microseconds and y-axis fixed from 300 K to 400 K. Text objects are added for annotating the current simulation time.

2.6 Animation Update Function

The animation function updates the temperature field and the corresponding temperature graphs by performing several time steps per animation frame.

```python
times = []
max_temps = []
avg_temps = []

t_current = 0.0

def init():
    im.set_data(T)
    line_max.set_data([], [])
    line_avg.set_data([], [])
    time_text_max.set_text('')
    time_text_avg.set_text('')
    return im, line_max, line_avg, time_text_max, time_text_avg

def animate(frame):
    global T, t_current, times, max_temps, avg_temps

    steps_per_frame = 10
    for _ in range(steps_per_frame):
        T = update_T(T, t_current)
        t_current += dt

    im.set_data(T)

    max_temp = np.max(T)
    avg_temp = np.mean(T)
    times.append(t_current * 1e6)
    max_temps.append(max_temp)
    avg_temps.append(avg_temp)
    line_max.set_data(times, max_temps)
    line_avg.set_data(times, avg_temps)

    time_text_max.set_text(f"t = {t_current * 1e6:.2f}  s ")
    time_text_avg.set_text(f"t = {t_current * 1e6:.2f}  s ")

    return im, line_max, line_avg, time_text_max, time_text_avg

frames = int(T_total / (dt * 10))
anim = animation.FuncAnimation(fig, animate, init_func=init, frames=frames,
                               interval=50, blit=True)

plt.tight_layout()
plt.show()
```

Listing 6: Animation update function.

Explanation: The `animate` function advances the simulation by performing multiple time steps per frame. It updates the temperature field and computes the maximum and average temperatures at each frame. The simulation time is converted to microseconds when plotting. The function updates the plot lines and time annotations accordingly. Finally, a `FuncAnimation` object is created to animate the simulation.

3 Complete Code With Comments

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import animation

# Domain and Grid Parameters
Lx, Ly = 1000.0, 1000.0  # domain in  m
nx, ny = 100, 100       # resolution in pixels
dx, dy = Lx/(nx-1), Ly/(ny-1)  #  m

# Convert spatial step to meters
dx_m, dy_m = dx * 1e-6, dy * 1e-6

# Material Properties
density = 7800.0         # kg/m^3
specific_heat = 500.0    # J/(kg*K)
conductivity = 50.0      # W/(m*K)

# Thermal diffusivity
alpha = conductivity / (density * specific_heat)  # ~1.28e-05 m^2/s

# Time-stepping Parameters
# CFL condition: dt <= (dx_m)^2 / (4*alpha)
dt = 0.25 * dx_m**2 / alpha
T_total = 0.1  # seconds
n_steps = int(T_total / dt)

# Heat Source Parameters

# PDE's source term: Q/(rho*c_p)
Q0 = 2e12              # amplitude in W/m^3
sigma = 100.0           # standard deviation of the Gaussian in  m
v = 1.5e6                # velocity in  m /s

# Heat source movement path; note X, Y are in  m  and t is in s.
def heat_source(X, Y, t):
    x_center = v * t
    y_center = Ly / 2     # y-axis fixed
    # Gaussian spatial profile
    return Q0 * np.exp(-(((X - x_center)**2 + (Y - y_center)**2) / (2 * sigma**2))
    )

# Initial Grid and Temperature Setup
x = np.linspace(0, Lx, nx)   #  m
y = np.linspace(0, Ly, ny)   #  m
X, Y = np.meshgrid(x, y)

# Initial temperature: 300K amgient
T_ambient = 300.0
T = T_ambient * np.ones((ny, nx))

# Finite Difference Temperature Update
def update_T(T, t):
    T_new = T.copy()

```

```python
    # Compute Laplacian on interior points (using dx_m and dy_m in meters)
    laplacian = (
        (T[1:-1, 2:] - 2*T[1:-1, 1:-1] + T[1:-1, 0:-2]) / dx_m**2 +
        (T[2:, 1:-1] - 2*T[1:-1, 1:-1] + T[0:-2, 1:-1]) / dy_m**2
    )

    # Compute source term (in K/s): Q/(rho*c_p)
    source = heat_source(X[1:-1, 1:-1], Y[1:-1, 1:-1], t) / (density *
    specific_heat)

    # Update temperature on interior points
    T_new[1:-1, 1:-1] = T[1:-1, 1:-1] + dt * (alpha * laplacian + source)

    # Dirichlet boundary conditions where T = T_ambient at boundaries
    T_new[0, :] = T_ambient
    T_new[-1, :] = T_ambient
    T_new[:, 0] = T_ambient
    T_new[:, -1] = T_ambient

    return T_new

# Animation Plot Setup
# Temperature field, max temperature, and average temperature subplots
fig, (ax_field, ax_max, ax_avg) = plt.subplots(1, 3, figsize=(18, 6))

# Temperature field plot
im = ax_field.imshow(T, extent=[0, Lx, 0, Ly], origin='lower', cmap='rainbow',
                        vmin=T_ambient, vmax=T_ambient + 50, animated=True)
ax_field.set_title("Temperature Distribution (K)")
ax_field.set_xlabel("x ( m )")
ax_field.set_ylabel("y ( m )")
cbar = plt.colorbar(im, ax=ax_field)
cbar.set_label("Temperature (K)")

# Graph of maximum temperature vs. time
line_max, = ax_max.plot([], [], 'b-', lw=2)
ax_max.set_xlim(0, 15000)  # x-axis from 0 to 100,000  s
ax_max.set_ylim(T_ambient, 400)  # y-axis from 300 K to 400 K
ax_max.set_title("Max Temperature vs Time")
ax_max.set_xlabel("Time ( s )")
ax_max.set_ylabel("Max Temperature (K)")
time_text_max = ax_max.text(0.1, 0.9, '', transform=ax_max.transAxes, fontsize=12)

# Graph of average temperature vs. time
line_avg, = ax_avg.plot([], [], 'r-', lw=2)
ax_avg.set_xlim(0, 15000)
ax_avg.set_ylim(T_ambient, 400)
ax_avg.set_title("Avg Temperature vs Time")
ax_avg.set_xlabel("Time ( s )")
ax_avg.set_ylabel("Average Temperature (K)")
time_text_avg = ax_avg.text(0.1, 0.9, '', transform=ax_avg.transAxes, fontsize=12)

times = []
max_temps = []
avg_temps = []

```

```python
t_current = 0.0

# Animation Initialization Function
def init():
    im.set_data(T)
    line_max.set_data([], [])
    line_avg.set_data([], [])
    time_text_max.set_text('')
    time_text_avg.set_text('')
    return im, line_max, line_avg, time_text_max, time_text_avg

# Animation Update Function
def animate(frame):
    global T, t_current, times, max_temps, avg_temps

    steps_per_frame = 10
    for _ in range(steps_per_frame):
        T = update_T(T, t_current)
        t_current += dt

    # Temperature field image update
    im.set_data(T)

    # Compute and store the maximum and average temperatures
    max_temp = np.max(T)
    avg_temp = np.mean(T)
    times.append(t_current * 1e6)
    max_temps.append(max_temp)
    avg_temps.append(avg_temp)
    line_max.set_data(times, max_temps)
    line_avg.set_data(times, avg_temps)

    # Time annotations update
    time_text_max.set_text(f"t = {t_current * 1e6:.2f}  s ")
    time_text_avg.set_text(f"t = {t_current * 1e6:.2f}  s ")

    return im, line_max, line_avg, time_text_max, time_text_avg

frames = int(T_total / (dt * 10))
anim = animation.FuncAnimation(fig, animate, init_func=init, frames=frames,
                               interval=50, blit=True)

plt.tight_layout()
plt.show()
```

Listing 7: Complete code.

4 Conclusion

We have presented a finite-difference model to simulate transient heat conduction with a moving Gaussian heat source. The simulation incorporates material properties and produces real-time visualizations of the temperature field and its key metrics. Future work may extend this approach to three-dimensional problems or incorporate additional physical effects.

Acknowledgments

The author thanks the supporting literature for valuable insights in developing this simulation framework.